# The Semester Project-VI
# File System on Block Device

This article, which is part of the series on Linux device drivers, enhances the previously written bare-bones file system module, to connect with a real hardware partition.

After writing the bare-bones file system, the first thing Pugs figured out was how to read from the underlying block device. The following is a typical way of doing it:

```
struct buffer_head *bh;

bh = sb_bread(sb, block); /* sb is the struct super_block pointer
*/
// bh->b_data contains the data
// Once done, bh should be released using:
brelse(bh);
```

To do the above, and various other real SFS (Simula File System) operations, Pugs felt he needed to have his own handle to be a key parameter, which he added as follows (in the previous *real_sfs_ds.h):*

```
typedef struct sfs_info {
    struct super_block *vfs_sb; /* Super block structure from VFS
for this fs */
    sfs_super_block_t sb; /* Our fs super block */
    byte1_t *used_blocks; /* Used blocks tracker */
    byte1_t block[SIMULA_FS_BLOCK_SIZE]; /* A block-size scratch
space */
} sfs_info_t;
```

The main idea behind this was to put all required static global variables in a single structure, and point to that by the private data pointer of the file system, which is the *s_fs_info* pointer in the *struct super_block* structure. With that, the key changes in the *fill_super_block* (in the previous *real_sfs_bb.c* file) become:

- Allocate the structure for the handle, using *kzalloc()*
- Initialise the structure for the handle (through *init_browsing()*)
- Read the physical super block, verify and translate the information from it into the VFS super block (through *init_browsing()*)
- Point to it by *s_fs_info* (through *init_browsing()*)